Campus Service Request Management System

Design, Technical, User and Testing Documentation

IS305 Object-Oriented Programming
AT3 Major Project (Pass, Credit and Distinction)
Divine Word University

Student: Mareerose Possi
Student ID: 240456

Technology: JavaScript (Node.js), classes only, JSON files, no database

1. Requirements Document

1.1 Purpose

The campus needs one simple system where students and staff report problems (broken Wi-Fi, a broken light, a spill) and where the service team can see, assign and finish the work. The old way (phone calls and paper) loses requests and nobody can see their progress. The Campus Service Request Management System is a console program that records every request, controls who may do what, and keeps a history of every change.

1.2 Users (roles)

Role	What this person does
Student or Staff Requester	Registers, submits requests, views their own requests, updates or cancels their own Submitted request, searches requests.
Service Officer	Reviews new requests, sets the priority, assigns a Technician, verifies finished work and closes the request.
Technician	Sees assigned requests, starts the work, adds progress notes and marks the request Resolved.
System Administrator	Looks at all requests, filters and sorts them, reads the history, audit log and management reports. (In this console program these are open menu options, not a separate login.)

1.3 Functional requirements

ID	Requirement	Level
FR1	Register a user with ID, name, email and type. Reject duplicate IDs and invalid data.	Pass
FR2	Submit a service request with title, description, location, category and priority.	Pass
FR3	Give each request a unique ID and the starting status Submitted.	Pass
FR4	View a request by ID, view my requests, view all requests.	Pass
FR5	Update my own request only while it is Submitted.	Pass
FR6	Cancel my own request only while it is Submitted. A Cancelled request cannot be cancelled again.	Pass
FR7	Search requests by ID or title and show a count of requests per status.	Pass
FR8	Four user types (Student, Staff, Service Officer, Technician) that inherit from User.	Credit
FR9	Request types for ICT, Maintenance, Cleaning (and General) with their own extra fields and validation.	Credit
FR10	Controlled workflow: Submitted, Reviewed, Assigned, In Progress, Resolved, Closed or Cancelled. Invalid jumps are rejected.	Credit
FR11	Role permissions: only a Service Officer reviews, assigns and closes; only the assigned Technician starts, updates and resolves.	Credit
FR12	Request history: every approved change records old status, new status, who, role, comment and date.	Credit
FR13	Filter by category, status, priority and Technician. Sort by date and priority.	Credit
FR14	Abstract-style base class: each request type calculates its own priority score, target resolution time and summary (polymorphism).	Distinction
FR15	Save and load users, requests, history and audit log in JSON files. Handle missing, empty or damaged files.	Distinction

ID	Requirement	Level
FR16	Rebuild the correct request class from saved data (factory).	Distinction
FR17	Audit log of important actions, including refused ones.	Distinction
FR18	Management reports: by status, category, priority, urgent, overdue, per Technician, completed, average resolution time, volume by location.	Distinction
FR19	Automated tests for the main rules and for file handling.	Distinction

1.4 Non-functional requirements

- **Easy to use:** numbered menus, clear SUCCESS and ERROR messages, choices picked from lists.
- **Reliable:** invalid data is rejected before it is stored or saved, and a program error never crashes the menu.
- **Simple to read:** small classes, comments, one job per class, no database and no extra packages.
- **Safe:** private fields, no passwords stored, simulated data only.
- **Portable:** runs with plain Node.js 20 or newer on any computer.

1.5 Business rules

- Allowed status changes: Submitted to Reviewed or Cancelled; Reviewed to Assigned; Assigned to In Progress; In Progress to Resolved; Resolved to Closed.
- Closed and Cancelled are final. Nothing can change them.
- Only a Student or Staff user can submit a request.
- Priorities are Low, Normal, High or Urgent. Categories are ICT Support, Facilities Maintenance, Cleaning and Sanitation, General Campus Service.
- A request is overdue when it is still Submitted, Reviewed, Assigned or In Progress and older than its target resolution time.

1.6 Assumptions

- One person uses the console at a time and types their own user ID (no passwords, as the brief says not to store them).
- A fourth category, General Campus Service, has its own small class so every category can be stored and processed.
- All data in the JSON files is simulated.

2. System Design and UML

2.1 Use case diagram

The diagram shows who uses the system and what each person can do.

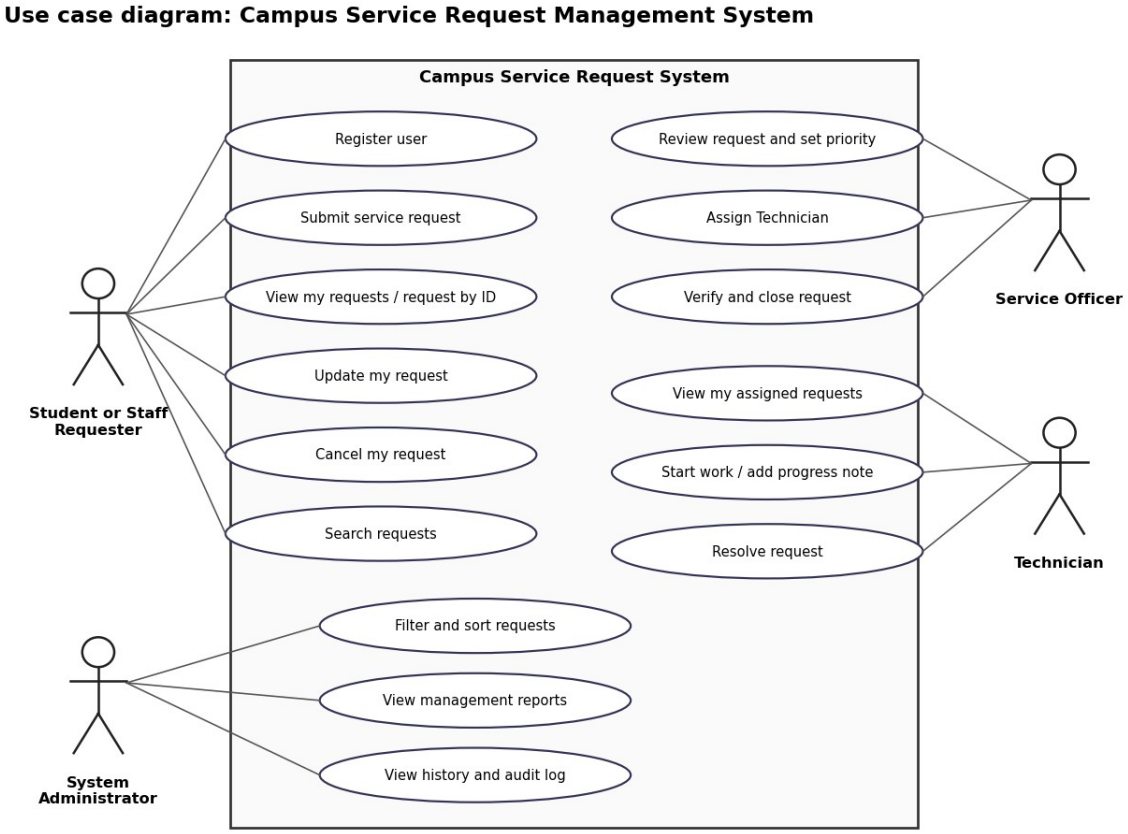


Figure 1: Use case diagram

2.2 Class diagram

Blue classes are users, orange classes are requests, green classes are the services that apply the rules, red classes handle JSON files and purple classes handle the console. A hollow triangle means inheritance. A dashed arrow means "uses". The diamond means the manager holds many users and requests. The italic methods in ServiceRequest are the abstract-style methods that each subclass must override.

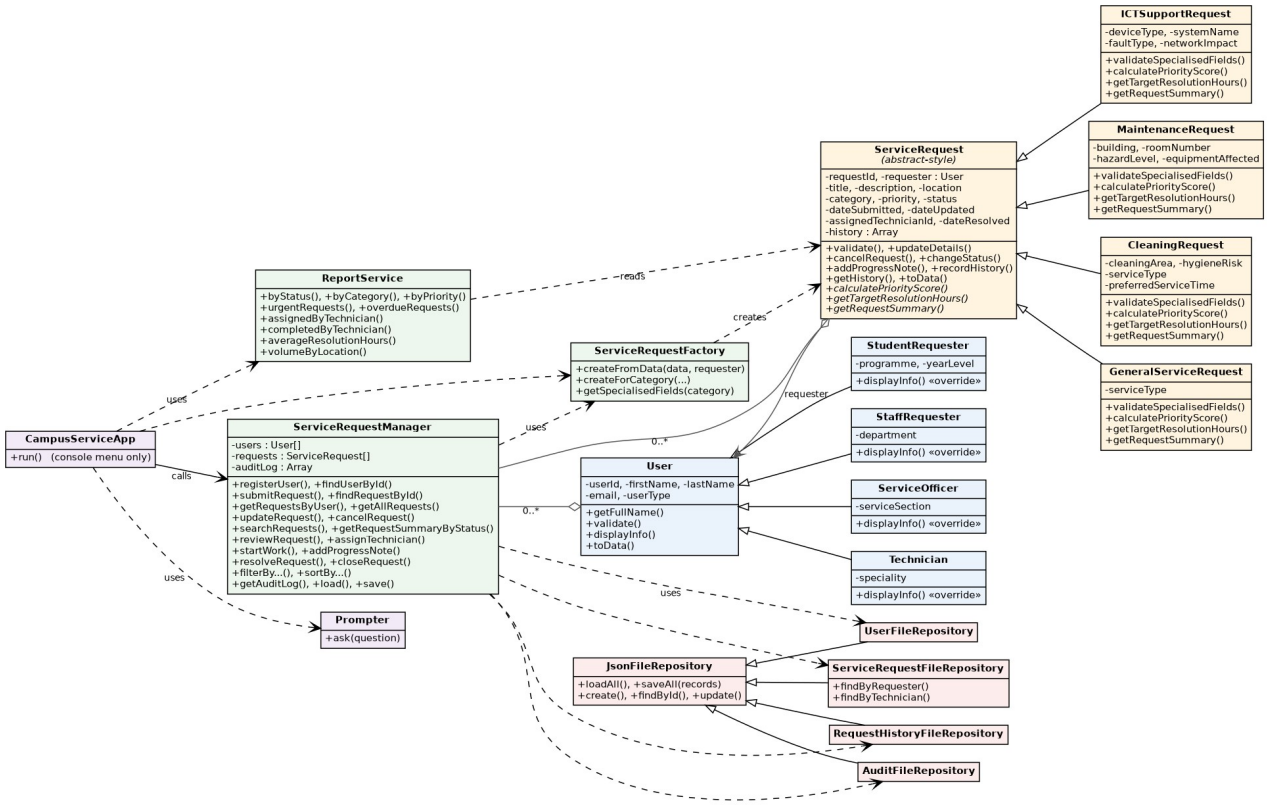


Figure 2: Class diagram (all classes)

2.3 OOP ideas used

Idea	Where it is used in this project
Encapsulation	Every class keeps its data in private # fields. Outside code uses getters and checked setters, so bad data cannot get in.
Inheritance	StudentRequester, StaffRequester, ServiceOfficer and Technician extend User. ICT, Maintenance, Cleaning and General requests extend ServiceRequest and call super().
Polymorphism	calculatePriorityScore(), getTargetResolutionHours() and getRequestSummary() are called the same way on every request but give a different answer for each type. The report and the menu never ask which type a request is.
Abstraction	ServiceRequest does not know how to score or summarise a request. It throws a clear error until a subclass provides the method.
Composition	A request holds a User object as its requester. The manager holds lists of users and requests.
Separation of concerns	The menu only talks to the person. The manager applies the rules. The repositories are the only code that touches files.

2.4 Status workflow

`Submitted > Reviewed > Assigned > In Progress > Resolved > Closed`

`Submitted > Cancelled` (Cancelled and Closed are final)

2.5 Sequence diagrams

The three diagrams show how the objects work together for the most important actions.

Sequence diagram 1: Submit a service request

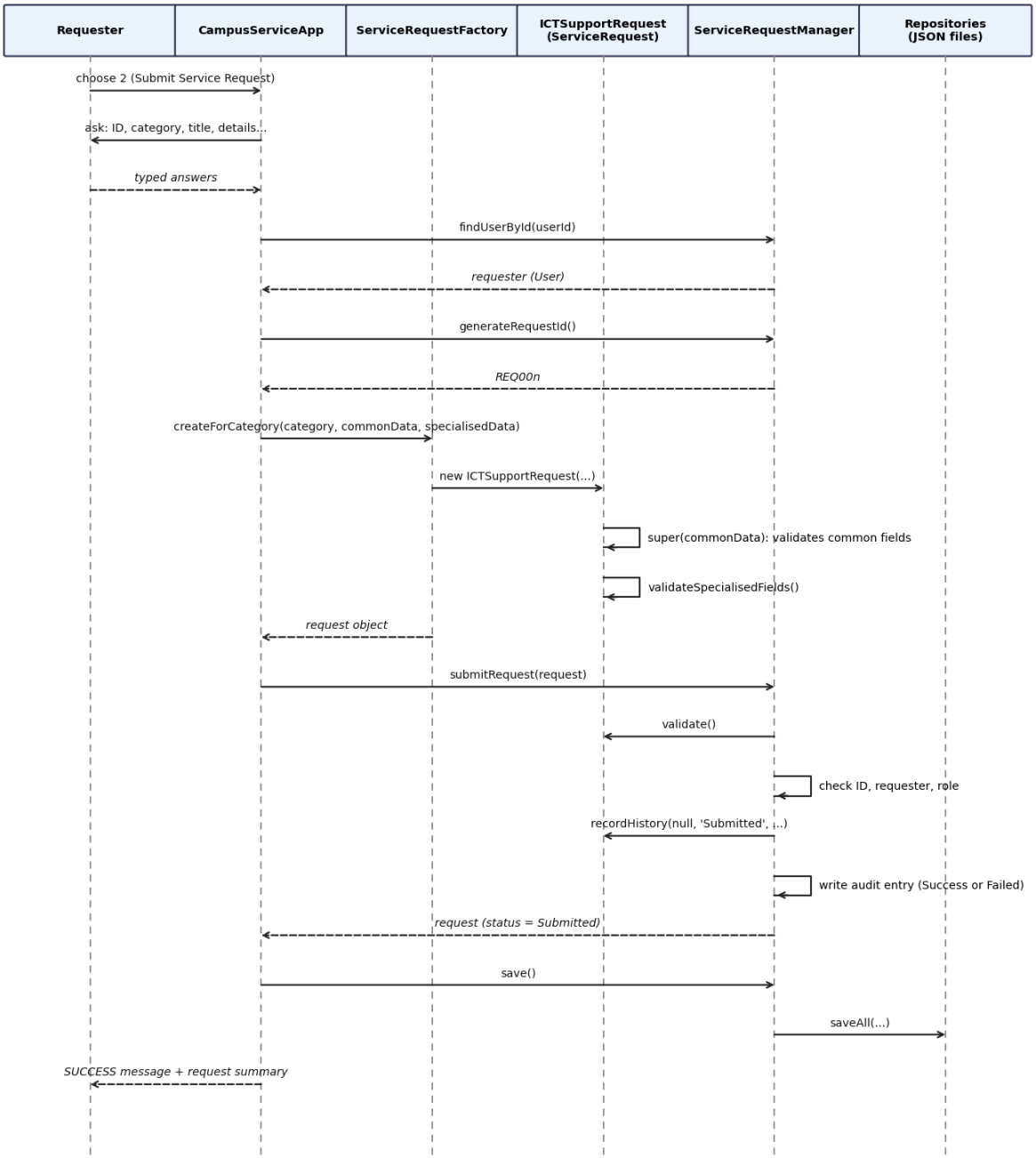


Figure 3: Submit a service request

Sequence diagram 2: Review a request and assign a Technician

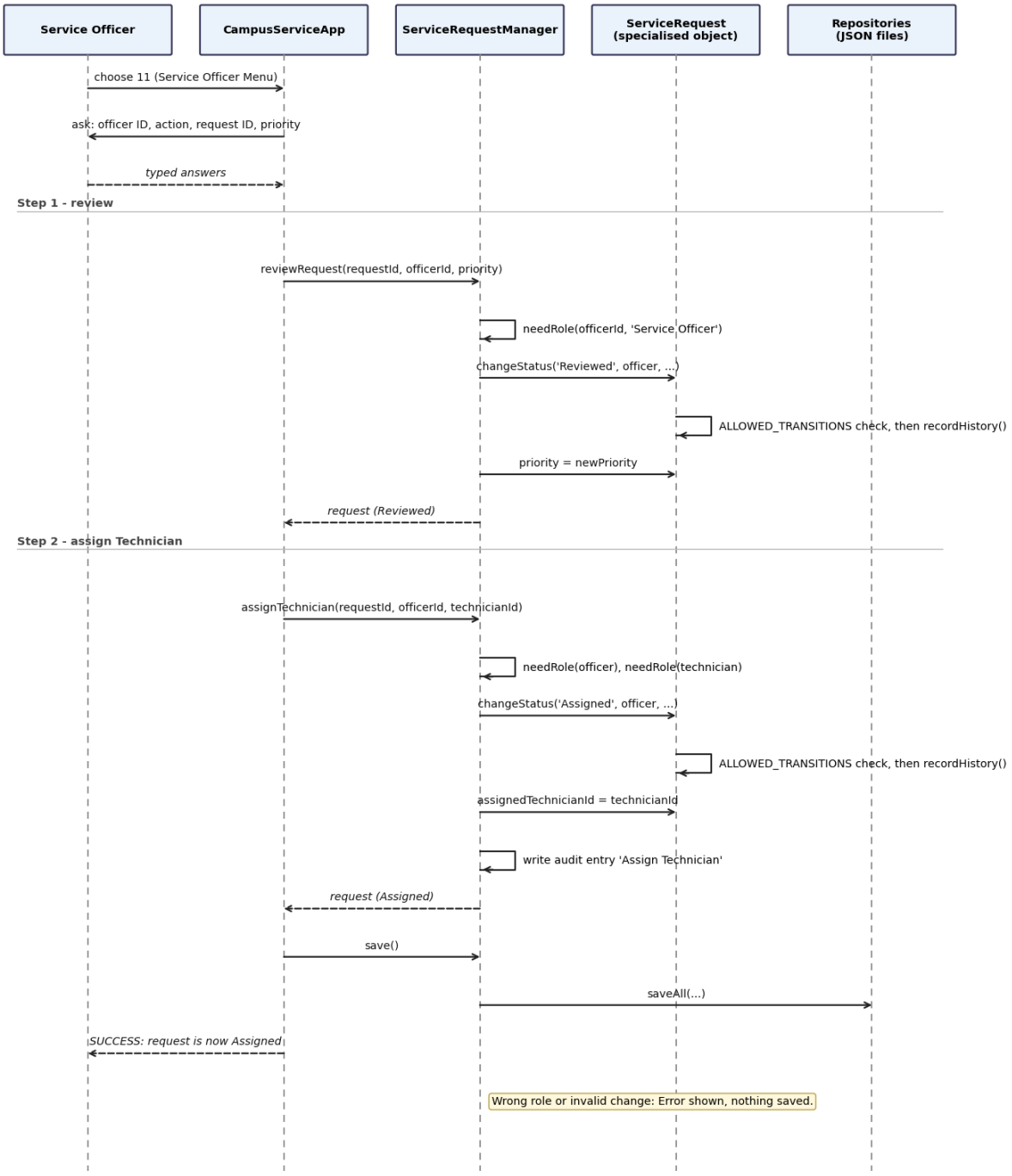


Figure 4: Review a request and assign a Technician

Sequence diagram 3: Resolve a request and close it

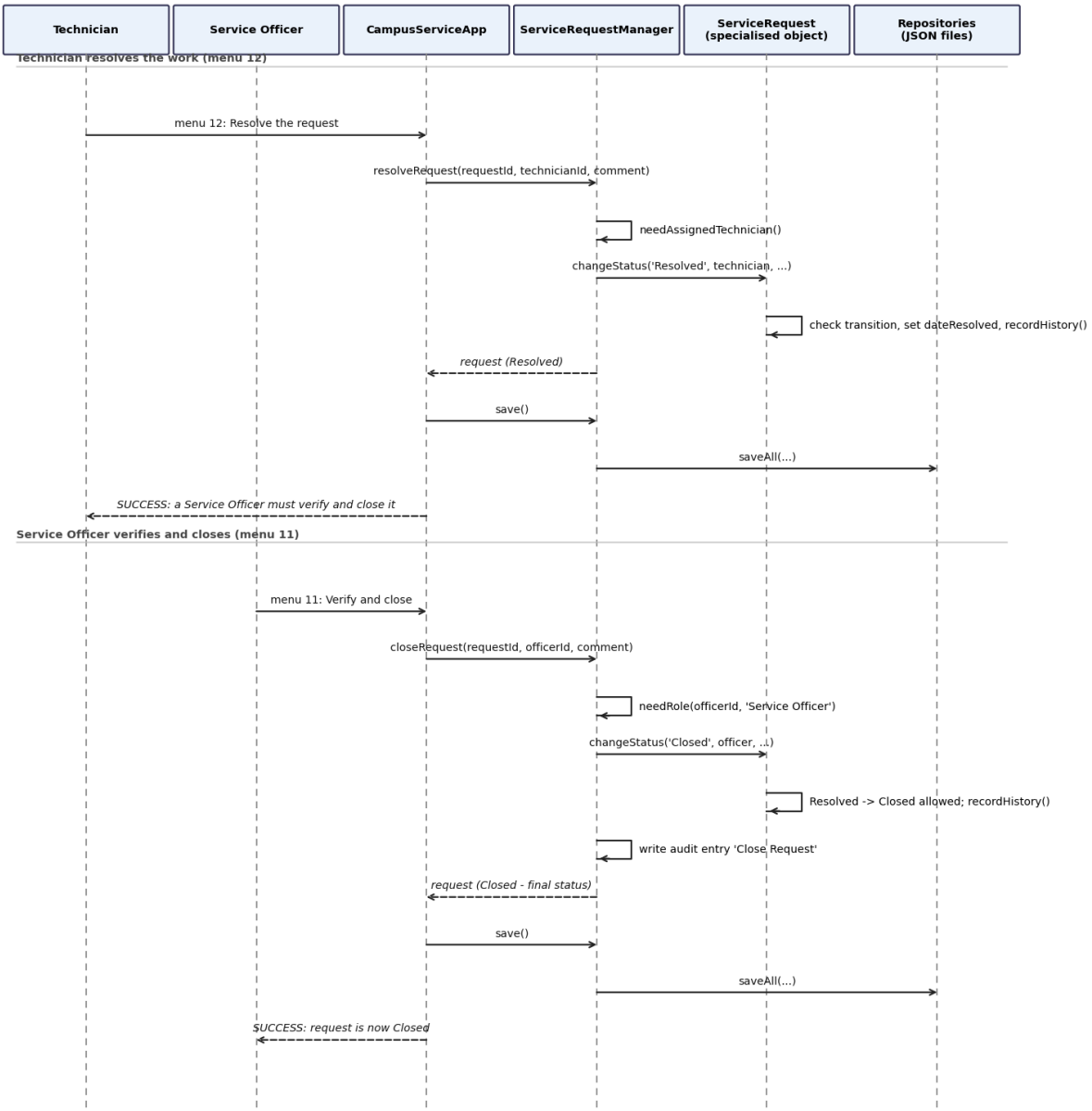


Figure 5: Resolve a request and close it

2.6 Data design (JSON files)

File	Holds	Main fields
data/users.json	All users	userId, firstName, lastName, email, userType + role field (programme and yearLevel, department, serviceSection or speciality)
data/serviceRequests.json	All requests	requestId, requestType, requesterId, title, description, location, category, priority, status, assignedTechnicianId, dateSubmitted, dateUpdated, dateResolved + the type's own fields
data/requestHistory.json	Every history entry	historyId, requestId, previousStatus, newStatus, action, actorId, actorRole, comment, dateTime
data/auditLog.json	Important actions	auditId, actorId, action, requestId, description, dateTime, result

3. Technical Documentation

3.1 Files and what each one does

File	Level	Job
src/User.js	Pass	User base class with validation, getters, controlled setters, getFullName(), validate(), displayInfo().
src/ServiceRequest.js	Pass to Distinction	Base request: validation, update, cancel, status workflow, progress notes, history, abstract-style methods.
src/ServiceRequestManager.js	Pass to Distinction	Holds the arrays and applies all rules: registration, submit, update, cancel, search, review, assign, start, resolve, close, filter, sort, audit, load and save.
src/CampusServiceApp.js	Pass to Distinction	Console menu. Asks questions and prints answers. Has no file code.
src/UserRoles.js	Credit	Four user subclasses and createUserFromData().
src/RequestTypes.js	Credit	Four request subclasses with their own fields, validation and overridden methods.
src/ServiceRequestFactory.js	Distinction	Creates a new request for a category, or rebuilds one from saved JSON data.
src/Repositories.js	Distinction	JsonFileRepository plus one repository for users, requests, history and audit. Uses fs/promises.
src/ReportService.js	Distinction	Nine management reports using filter, map, reduce and sort.
src/Prompter.js	Helper	Reads typed lines (also works with piped input).
tests/campus.test.js	Distinction	40 automated tests using the built-in node:test runner.
scripts/seedData.js	Helper	Creates simulated demo data by running the real workflow.

3.2 Important design decisions

- **Status can only change through changeStatus().** There is no public status setter, so the allowed-transition table cannot be bypassed.
- **Rules are checked before anything changes.** For example reviewRequest() checks the priority first, so a failed action never leaves a half-changed request.
- **The manager works in memory; the repositories handle files.** The menu calls manager.save() after each successful change. If saving fails the menu shows a warning and keeps running.
- **Dates come from a clock function.** The manager and reports accept a clock, so tests can check overdue and average-time results without waiting.
- **History is stored once.** Each request keeps its history in memory; requestHistory.json stores the rows and the factory puts them back on load.
- **Failed actions are audited but not added to a request's history.** History shows approved changes only; the audit log shows both successes and refusals.

3.3 Priority score and target time

Each type turns the priority (Low 1, Normal 2, High 3, Urgent 4) into a score by multiplying by 10 and adding points for its own risk field. The target resolution time also depends on that field.

Request type	Extra points added to the score	Target hours
ICTSupportRequest	Network impact: None 0, Local 10, Campus-Wide 25	None 48, Local 24, Campus-Wide 4

Request type	Extra points added to the score	Target hours
MaintenanceRequest	Hazard level: Low 0, Medium 10, High 30	Low 72, Medium 24, High 8
CleaningRequest	Hygiene risk: Low 0, Medium 8, High 20	Low 48, Medium 24, High 6
GeneralServiceRequest	None	72

3.4 Error handling

- Every invalid action throws an Error with a plain message. The menu catches it, prints "ERROR: ..." and shows the menu again.
- A missing data file is created as an empty array. An empty file counts as no records.
- A damaged JSON file stops the program at start with a clear message that names the file.
- A failed save prints a warning; the data stays in memory.

3.5 Menu map

Menu	Action	Method used
1	Register User	registerUser()
2	Submit Service Request	ServiceRequestFactory.createForCategory(), submitRequest()
3 / 4 / 5	View by ID / my requests / all	findRequestById(), getRequestsByUser(), getAllRequests()
6 / 7	Update / Cancel my request	updateRequest(), cancelRequest()
8 / 9	Search / Summary by status	searchRequests(), getRequestSummaryByStatus()
10	Exit	-
11	Service Officer menu	reviewRequest(), assignTechnician(), closeRequest()
12	Technician menu	startWork(), addProgressNote(), resolveRequest(), filterByTechnician()
13	Filter and sort	filterBy...(), sortByDateSubmitted(), sortByPriority()
14	Management reports	ReportService methods
15	History, audit log, users	getRequestHistory(), getAuditLog(), getAllUsers()

4. User Guide

4.1 Start the program

- Install Node.js 20 or newer.
- Open a terminal in the project folder.
- Optional: type npm run seed to load simulated demo data.
- Type npm start to open the menu. Type a number and press Enter.

Demo IDs after seeding: students DWU2026001 and DWU2026002, staff STF001, Service Officer OFF001, Technicians TECH001 and TECH002.

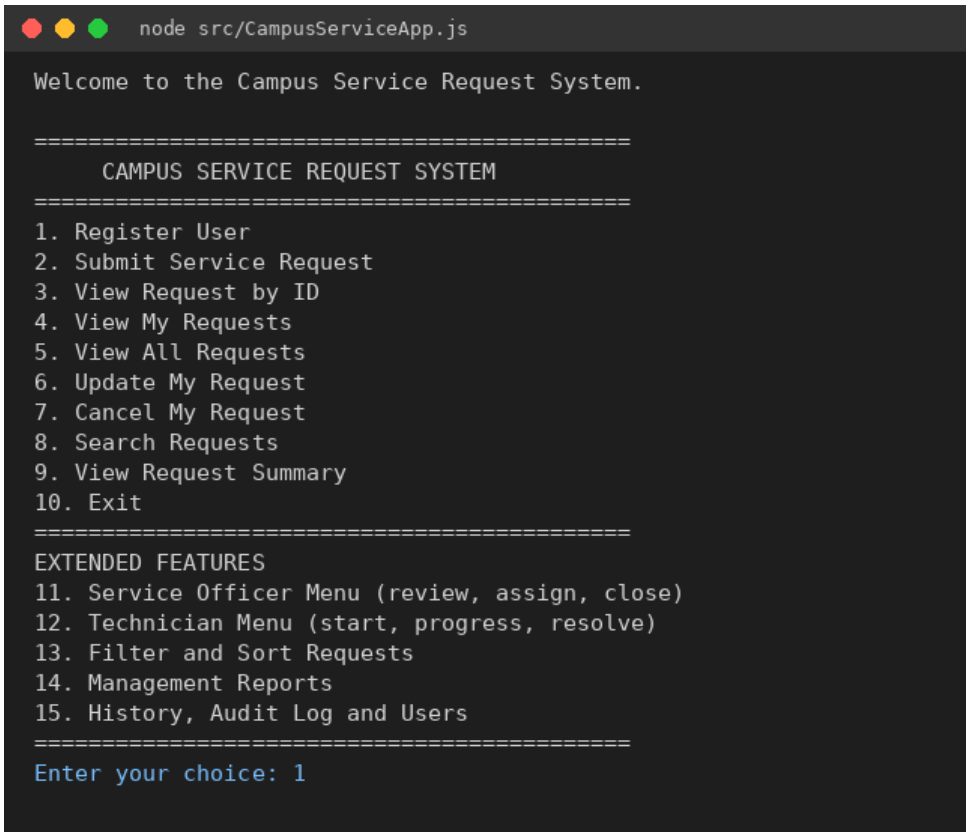


Figure 6: The main menu

4.2 Register a user (menu 1)

Type a new ID, your name and email, choose your user type from the list, then answer the one question for that type. A duplicate ID is refused.

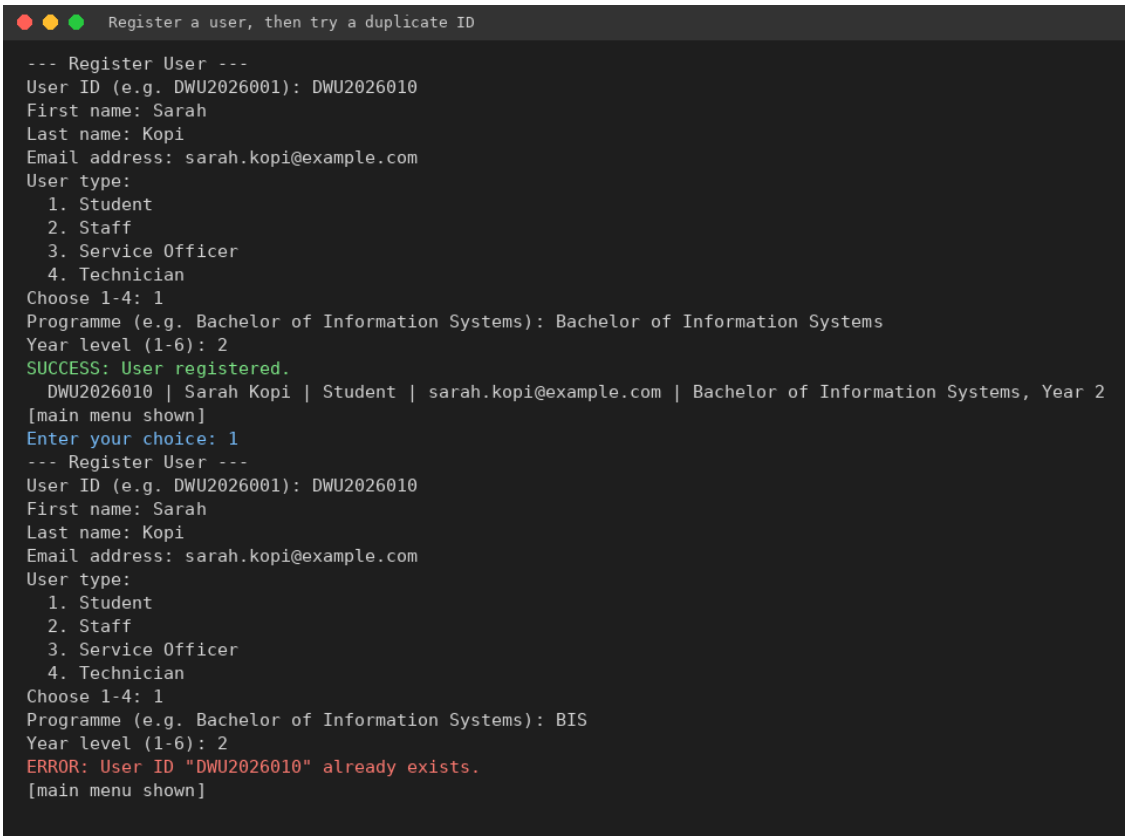


Figure 7: Registering a user, and the error for a duplicate ID

4.3 Submit a request (menu 2)

Enter your user ID, pick a category, type the title, description and location, pick a priority, then answer the extra questions for that category. The system gives the request an ID such as REQ006 and shows a summary with its priority score and target time.

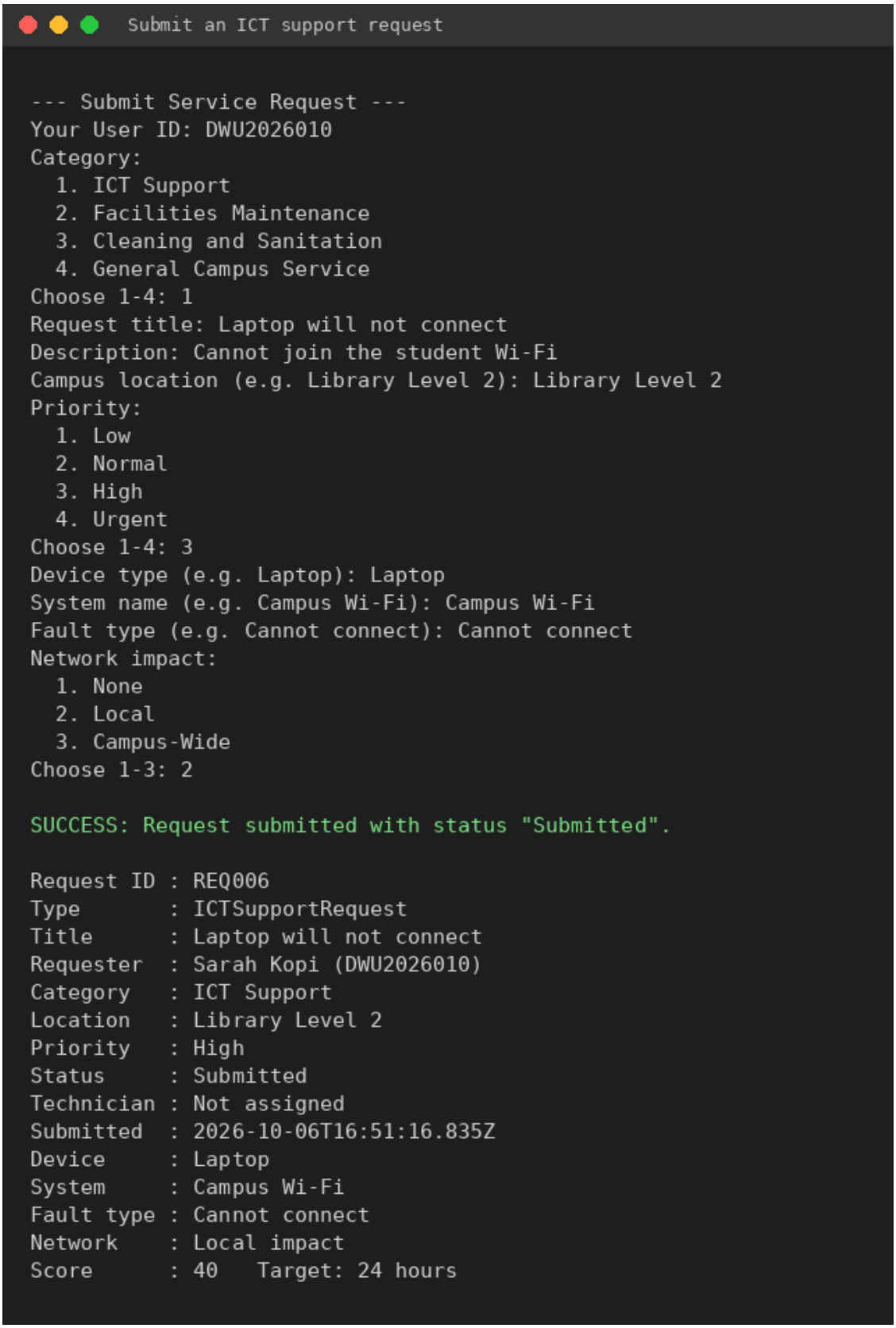


Figure 8: Submitting an ICT support request

4.4 View, update, cancel and search (menus 3 to 9)

- **3:** type a request ID to see all its details.
- **4:** type your user ID to list your own requests.
- **5:** list every request.

- **6:** type your user ID and the request ID. Press Enter to keep a value. Works only while the status is Submitted and only on your own request.
- **7:** cancel your own Submitted request.
- **8:** type part of a request ID or title.
- **9:** see how many requests are in each status.

4.5 Service Officer and Technician (menus 11 and 12)

A Service Officer reviews a Submitted request and sets its priority, assigns a Technician, and later verifies and closes a Resolved request. The assigned Technician starts the work, adds progress notes and resolves it. The picture shows a Student trying to review a request (refused) and a wrong Technician trying to start work (refused), followed by the correct people succeeding.

Officer and Technician workflow (with a permission error)

```
[main menu shown]
Enter your choice: 11
Your Service Officer ID: DWU2026001
Officer action:
  1. Review a request and set its priority
  2. Assign a Technician
  3. Verify and close a Resolved request
Choose 1-3: 1
Request ID: REQ006
Priority:
  1. Low
  2. Normal
  3. High
  4. Urgent
Choose 1-4: 2
ERROR: Only a Service Officer can review requests.
[main menu shown]
Enter your choice: ERRX
Please choose a number from the menu.
[main menu shown]
Enter your choice: 11
Your Service Officer ID: OFF001
Officer action:
  1. Review a request and set its priority
  2. Assign a Technician
  3. Verify and close a Resolved request
Choose 1-3: 1
Request ID: REQ006
Priority:
  1. Low
  2. Normal
  3. High
  4. Urgent
Choose 1-4: 3
SUCCESS: REQ006 is now Reviewed with priority High.
[main menu shown]
Enter your choice: 11
Your Service Officer ID: OFF001
Officer action:
  1. Review a request and set its priority
  2. Assign a Technician
  3. Verify and close a Resolved request
Choose 1-3: 2
Request ID: REQ006
Technician ID: TECH001
SUCCESS: REQ006 is now Assigned to TECH001.
[main menu shown]
Enter your choice: 12
Your Technician ID: TECH002
Technician action:
  1. View my assigned requests
  2. Start work
  3. Add a progress note
  4. Resolve the request
Choose 1-4: 2
Request ID: REQ006
ERROR: Only the assigned Technician can start the work.
[main menu shown]
Enter your choice: 12
Your Technician ID: TECH001
Technician action:
  1. View my assigned requests
  2. Start work
  3. Add a progress note
  4. Resolve the request
Choose 1-4: 2
Request ID: REQ006
SUCCESS: REQ006 is now In Progress.
[main menu shown]
```

Figure 9: Officer and Technician workflow with permission errors

4.6 Filter, reports and history (menus 13 to 15)

- **13:** filter by category, status, priority or Technician, or sort by date or priority.
- **14:** nine management reports and a polymorphism demo.
- **15:** see the history of one request, the audit log, or the list of users.

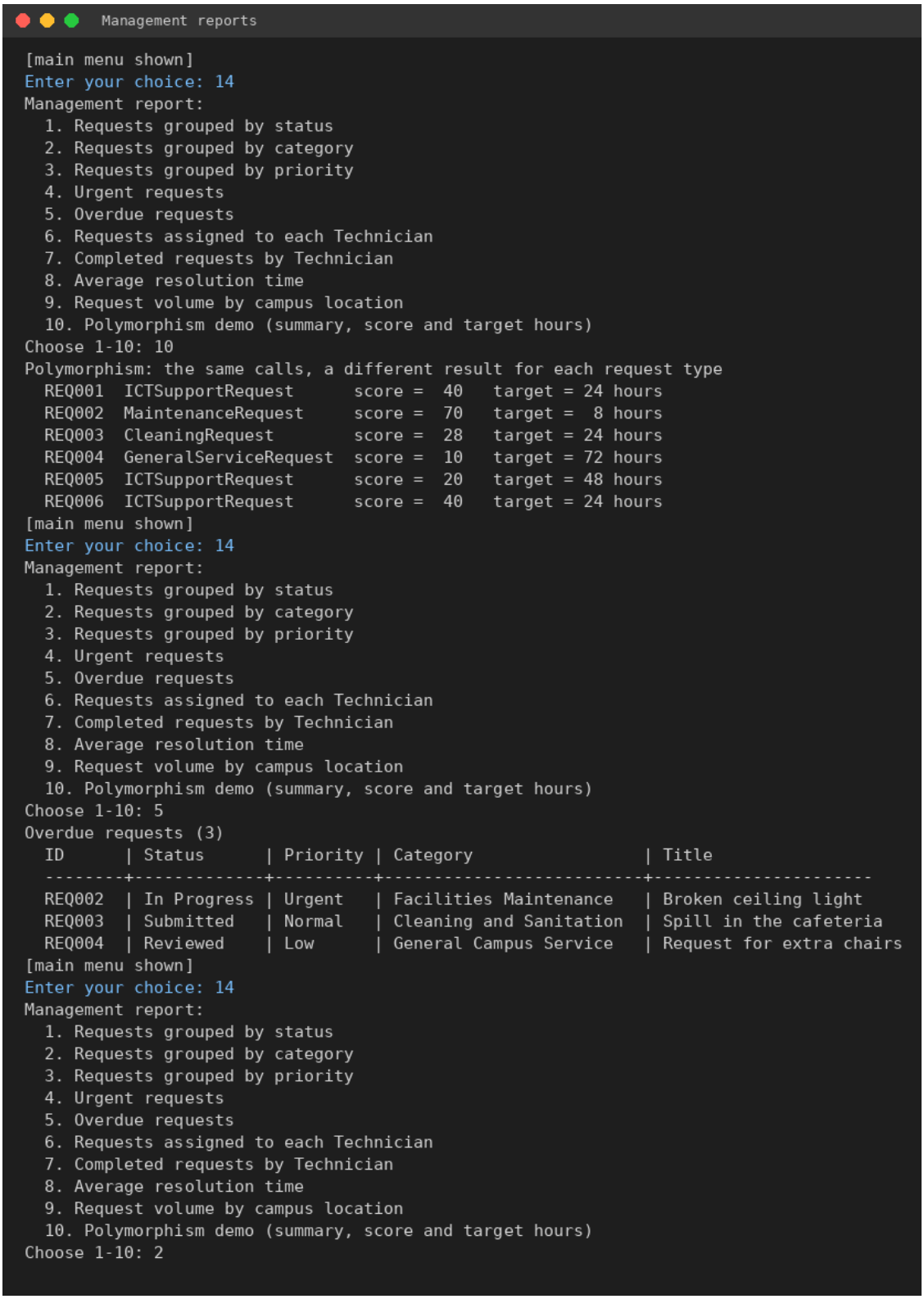


Figure 10: Polymorphism demo and overdue report

4.7 If something goes wrong

Message	What it means and what to do
User "X" is not registered	Use menu 1 to register first, or check the ID.
You can only update/cancel your own requests	Use the ID of the person who submitted the request.
Only Submitted requests can be updated	The request has already moved on and can no longer be edited.
Invalid status change: A -> B	The workflow does not allow that jump. Do the earlier step first.
Only the assigned Technician can …	Use the ID of the Technician the request was assigned to.
File read error … not valid JSON	A data file is damaged. Fix it or delete it (it will be re-created empty).

5. Test Report

5.1 Summary

Tests run: **40** Passed: **40** Failed: **0**

The tests use the built-in Node.js test runner (npm test). Every test that touches files uses a temporary folder, so the real data folder is never changed. Results below come from the last run of the full suite.

Level	Tests	Passed
Pass	15	15
Credit	11	11
Distinction	14	14

5.2 Test cases and results

ID	Level	Test	Input / action	Expected result	Result
T01	Pass	Create a valid User	User('U1','Mary','Kila','mary@example.com','Student')	Values saved and trimmed; getFullName() = Mary Kila	Passed
T02	Pass	Reject invalid User values	Empty ID, empty names, email 'not-an-email', type 'Visitor'	An Error with a clear message for each	Passed
T03	Pass	Encapsulation	Set email to 'bad', then to a valid email	Bad email rejected; valid email saved; no public data fields	Passed
T04	Pass	Register a valid user	registerUser(valid user)	User stored and found by ID	Passed
T05	Pass	Duplicate user ID	Register a second user with ID S1	Error 'already exists'; user count unchanged	Passed
T06	Pass	Submit a valid request	submitRequest(valid ICT request)	Stored with status Submitted	Passed
T07	Pass	Duplicate request ID	Submit REQ001 twice	Second one rejected	Passed
T08	Pass	Invalid category / priority	Category 'Gardening'; priority 'Whenever'	Error 'Unsupported category / priority'	Passed
T09	Pass	Missing title / description	Empty title; blank description	Error 'is required'	Passed
T10	Pass	View my requests	getRequestsByUser('S1') and ('S2')	Each user sees only their own requests	Passed
T11	Pass	Cancel a Submitted request	cancelRequest('REQ001','S1')	Status becomes Cancelled	Passed
T12	Pass	Cancel an already Cancelled request	Cancel REQ001 twice	Error 'already Cancelled'	Passed
T13	Pass	Ownership rule	User S2 updates / cancels S1's request	Error 'only update/cancel your own'; request unchanged	Passed
T14	Pass	Update own Submitted request	Update title and priority; then update after Reviewed	Update works first; rejected once Reviewed	Passed

ID	Level	Test	Input / action	Expected result	Result
T15	Pass	Search and status summary	Search 'wi-fi', 'req002', blank; getRequestSummaryByStatus()	Correct matches; blank gives none; correct counts	Passed
T16	Credit	Constructor chaining with super()	Create each subclass	userType / category / ID set by the parent constructor	Passed
T17	Credit	Specialised field validation	networkImpact 'Huge', empty device type, bad hazard level, bad service time	Error naming the field and allowed values	Passed
T18	Credit	Officer-only review and assign	Student, Technician then Officer try to review / assign	Only the Service Officer succeeds	Passed
T19	Credit	Assigned Technician only	Wrong Technician and Officer try start / note / resolve	Only the assigned Technician succeeds	Passed
T20	Credit	Invalid status transitions	Submitted -> Assigned, Submitted -> Closed, Assigned -> Resolved / Cancelled	Error 'Invalid status change'; status unchanged	Passed
T21	Credit	Final statuses and closing	Technician closes; Officer closes; act on Closed request	Only Officer closes; Closed is final	Passed
T22	Credit	Request history	Full workflow with a progress note	7 entries in order with status, actor, role, comment, date	Passed
T23	Credit	Failed actions not in history	Invalid close; non-officer review	History still has 1 entry	Passed
T24	Credit	Specialised summaries	getRequestSummary() for ICT, Maintenance, Cleaning	Each shows only its own extra details	Passed
T25	Credit	Filter and sort	Filter by category, status, priority, Technician; sort by priority	Correct lists and order	Passed
T26	Credit	Sort by date	sortByDateSubmitted(false / true)	Oldest first / newest first	Passed
T27	Distinction	Polymorphism	Score and target hours for 4 request types	Scores 40, 70, 28, 10; hours 24, 8, 24, 72	Passed
T28	Distinction	Abstract-style base class	Call the 3 abstract methods on a plain ServiceRequest	Clear 'must implement' errors	Passed
T29	Distinction	Save to JSON	save() into a temporary folder	4 files with the correct records, no passwords	Passed
T30	Distinction	Load from JSON	save(), then load() in a new manager	Correct subclasses restored; rules still work	Passed
T31	Distinction	Missing files	load() with no data files	Empty arrays; files created	Passed
T32	Distinction	Empty file	users.json contains only spaces	Treated as no records	Passed
T33	Distinction	Read errors	Broken JSON; JSON that is not an array	Clear 'File read error' message	Passed
T34	Distinction	Write errors	Save into a path that is really a file	Clear 'File write error' message	Passed

ID	Level	Test	Input / action	Expected result	Result
T35	Distinction	Repository methods	create, findById, findByRequester, findByTechnician, update	Correct results; duplicate and missing IDs rejected	Passed
T36	Distinction	Invalid data not saved	Duplicate user, empty title, unregistered requester	Rejected; saved files hold only valid records	Passed
T37	Distinction	Factory errors	Unknown request type; missing data; missing requester	Clear errors; valid data restored	Passed
T38	Distinction	Audit log	Success, denied action and priority change	Entries with ID, actor, action, request, time, result	Passed
T39	Distinction	Report calculations	All 9 reports on a known data set	Counts, urgent, overdue, per-Technician, 6 hour average, volume all correct	Passed
T40	Distinction	No resolved requests	Average resolution time with nothing resolved	Returns 0	Passed

5.3 Manual checks

The console menu was also run by hand with typed input. The screenshots in the user guide are from those runs. Checked: menu choice that is not a number, wrong user ID, duplicate user ID, permission refusals, a full request from Submitted to Closed, restart of the program with the saved data, and the reports on the seeded data.